The complete roadmap: From studying Instagram to building your own production-grade social platform

Your goal is to take an existing production system, understand how it works, independently design a comparable system, and implement it with production-quality architecture, reliability, security, and maintainability.

The best way to do this is to follow a structured, evidence-driven reconstruction process. You should never jump straight from looking at screens to writing code.

Let’s use Instagram as the reference example.

Phase 1 — Study the reference

Observe behavior, features, workflows, edge cases and limitations.

Phase 2 — Reconstruct the specification

Capabilities, domain model, state machines, contracts and requirements.

Phase 3 — Design your system

Architecture, data ownership, APIs, consistency, security and operations.

Phase 4 — Build incrementally

Implement complete vertical slices, validating each one before expanding.

Phase 5 — Production hardening

Failure testing, security, performance, deployment, monitoring and recovery.

Phase 6 — Release and operate

Deploy, validate against the specification, measure and improve.

There are two principles behind the entire process:

- Research from the outside inward: Start with observable product behavior and progressively infer the domain and architecture.
- Build from the inside outward: Start with foundational domain rules and data guarantees, then add application workflows, interfaces, infrastructure and UI.

You do not need to discover Instagram’s actual private architecture. You need to reconstruct its observable behavior and independently engineer a system that delivers the required capabilities and quality.

Your end-to-end development journey

Phase 0 — Define exactly what you are rebuilding

Before opening your editor, define the reconstruction target.

For the Instagram example, don't start with "rebuild all of Instagram." It is too large to understand and implement in one project.

Instead, define a product such as:

> *Build an independently implemented, production-oriented photo and video social platform inspired by Instagram's publicly observable capabilities.*

Decide the initial scope.

Core scope for your first version

Accounts

Registration, login, sessions, password reset

Profiles

Profile details, privacy and account settings

Posts and media

Upload photos, captions, edit and delete

Home feed

Posts from followed accounts, pagination

Social graph

Follow, unfollow, likes and comments

Notifications

In-app notifications and unread state

Search and discovery

User search, post discovery and hashtags

Direct messaging

Conversations, messages and read state

Stories

Temporary media, expiry and viewers

Advanced scale

Recommendation ranking, multi-region and massive traffic

6 capabilities selected. Use this as your initial scope checklist, not as a claim that these features have been fully specified.

Copy scope

Also define your assumptions:

- Expected users and traffic.
- Target platform: web, mobile or both.
- Required reliability and security.
- Available time and infrastructure budget.
- Features explicitly excluded from the first release.

Deliverable: `project-charter.md`, containing goals, scope, non-goals, constraints and initial success criteria.

Exit condition: You can explain what you are building and what you are deliberately not building.

Phase 1 — Study the reference product

Goal: discover what the system does, not merely what it looks like.

Use the product normally and document every important workflow. Use your own account or authorized access, and do not bypass access controls or inspect private user data.

For Instagram, investigate the following:

Area	What to investigate
Accounts	Signup, login, logout, password recovery
Profiles	Editing, privacy, public/private accounts
Posts	Creation, editing, deletion, media upload
Feed	Ordering, refresh, pagination, empty states
Following	Follow requests, unfollowing, follower lists
Engagement	Likes, unlike, comments, comment deletion
Notifications	Triggering events, unread state, navigation
Search	Query behavior, results and filtering
Moderation	Reporting, blocking, restricted interactions
Settings	Privacy, account controls and preferences

For every capability, investigate:

1. What action does the user perform?
2. What preconditions are required?

3. What changes in the system?
4. What does the user see afterward?
5. What happens when the action fails?
6. What changes for other users?
7. What happens on refresh, logout or another device?
8. What permissions or privacy rules apply?

For example, don't document merely "the Like button works."

Investigate:

- Can a user like their own post?
- What happens when they like and immediately unlike?
- Does the count update optimistically?
- What happens if the network fails?
- Does refreshing preserve the correct count?
- Can a private account's post be liked by an unauthorized user?

Mark findings as observed, inferred or unknown. Never treat a guess about the reference's internal implementation as a fact.

Deliverables:

- Feature inventory.
- User journeys.
- Screenshots and behavior notes where permitted.
- Edge-case inventory.
- Open questions and hypotheses.

Exit condition: You have mapped the important capabilities and their observable behavior, including failure cases.

Phase 2 — Turn observations into formal requirements

Now convert your research into an actual specification.

Organize requirements into three categories.

Functional requirements

What the system must do: publish posts, follow users, display a feed, upload media and send notifications.

Quality requirements

Security, correctness, latency, availability, durability, scalability, accessibility and maintainability.

System constraints

Budget, technology choices, expected load, deployment environment, external dependencies and delivery timeline.

Write testable requirements rather than vague statements.

Weak:

> *The feed should be fast and reliable.*

Better:

> *The feed must support cursor-based pagination, must not return posts from accounts the viewer is not authorized to see, and must have a defined latency target under a specified representative load.*

For every requirement, capture:

- A unique ID.
- Description and priority.
- Preconditions and expected result.
- Relevant authorization rules.
- Failure behavior.
- Acceptance tests.
- Evidence source and confidence.

Deliverables: `requirements.md`, acceptance criteria and a prioritized backlog.

Exit condition: Each in-scope feature has clear, testable behavior.

Phase 3 — Reconstruct the domain model

Now identify the concepts the business operates on.

For our example:

User

Identity and account state

Profile

Public-facing information

Follow

Directed social relationship

Post

Published content and lifecycle

MediaAsset

Uploaded file and processing state

Like

User engagement

Comment

Discussion on a post

Notification

User-facing event

Block

Restricted relationship

FeedEntry

A feed projection or materialized result

Illustrative domain concepts. Some may be combined, split or implemented as projections depending on the requirements.

For every entity, define:

- Identity and lifecycle.
- Fields and relationships.
- Who owns and modifies it.
- Business invariants.
- State transitions.
- Authorization policies.
- Data retention and deletion behavior.

Example invariants:

```
A post must have an authorized owner.

A user cannot follow themselves.

A user can have at most one active follow relationship
with a given target account.

A blocked relationship must prevent the interactions
specified by the privacy policy.

A media asset cannot be published until it satisfies
the required upload and processing rules.
```

Then model important state machines.

```
Post lifecycle

DRAFT → PROCESSING → PUBLISHED
            |
            └─────→ FAILED

PUBLISHED → DELETED
```

This is illustrative; the exact transitions depend on your specification.

Next, identify aggregate and transaction boundaries. For example, creating a post and persisting its initial metadata may require an atomic database transaction. Sending notifications and generating feed projections generally should not require one giant distributed transaction.

Deliverables: Entity relationship diagram, domain glossary, invariants, state machines, and a context map showing domain ownership.

Exit condition: You can explain what the core entities mean and which rules must always hold.

Phase 4 — Design the system architecture

Only now should you make major architecture decisions.

Start with a deployment-neutral component diagram:

```
Web / Mobile Client
        |
        v
     API Layer
        |
        v
  Modular Application
   |       |        |
 Identity Social   Content
   |       |        |
   +-------+--------+
           |
     PostgreSQL
           |
      Outbox Events
           |
      Background Jobs
       /     |      \
    Media  Feed   Notifications
       \     |      /
        Object Storage
```

This is a starting hypothesis, not a claim about Instagram's actual internals.

For your first version, a modular monolith with a relational database, object storage and a background worker is a reasonable default. Add a dedicated queue, cache, search engine or separate services when requirements and measurements justify them.

Decide and document:

1. Which modules own which domain concepts?
2. What can each module access or modify?
3. What operations require database transactions?
4. Which operations are synchronous?
5. Which side effects are asynchronous?
6. What is the source of truth for each kind of data?
7. What happens when a dependency fails?
8. How do components authenticate and authorize requests?
9. How will the system be deployed, monitored and recovered?

Create architecture decision records (ADRs) for consequential choices, including alternatives and trade-offs.

Deliverables: Context diagram, component diagram, deployment diagram, sequence diagrams, ADRs and explicit consistency boundaries.

Exit condition: You can trace a major user action through the system and explain its data ownership, failure behavior and dependencies.

Phase 5 — Design APIs and data contracts

Before implementing features, define the contracts between clients, application modules, databases and asynchronous consumers.

For example:

```
POST /api/v1/posts
Authorization: Bearer <token>
Idempotency-Key: <unique-key>
Content-Type: application/json
```

Define the request, response, errors, authorization and concurrency behavior.

A contract should cover:

- Input validation and output schemas.
- Authentication and resource-level authorization.
- Pagination, sorting and filtering.
- Stable error codes and HTTP status codes.
- Idempotency for operations where retries must not duplicate effects.
- Concurrency semantics where simultaneous changes are possible.

- Rate limits and resource limits.
- API versioning and backward compatibility.

For data, determine the authoritative schema, foreign keys, unique constraints, indexes, transaction boundaries and migration strategy.

Example:

```
Follow
-----
follower_id
followee_id
created_at

Constraints:
- follower_id ≠ followee_id
- unique(follower_id, followee_id)
- foreign keys reference valid accounts
```

A uniqueness constraint is important because two concurrent requests can otherwise create duplicate relationships even if your application checks for duplicates first.

For media uploads, define a separate lifecycle and storage contract rather than putting large file contents directly into ordinary relational rows.

Deliverables: OpenAPI specification, database schema, event contracts and migration plan.

Exit condition: The frontend, backend and persistence implementation can be developed against explicit contracts instead of inventing them independently.

Phase 6 — Plan implementation order

Do not implement every domain at once. Establish dependencies and choose complete vertical slices.

A sensible order for the example is:

1

Engineering foundation

Repository, configuration, CI, database migrations, logging, error handling and test infrastructure.

2

Identity and profiles

Registration, login, session lifecycle, authorization and profile management.

3

Media and posts

Secure upload, metadata, processing lifecycle, publish, edit and delete.

4

Social graph

Follow/unfollow, privacy checks, blocking and relationship queries.

5

Engagement

Likes, comments, uniqueness constraints and concurrent updates.

6

Home feed

Authorized post retrieval, ordering, cursor pagination and refresh behavior.

7

Notifications

Event-driven notifications, retries, deduplication and unread state.

8

Search and discovery

Search contracts, indexing, privacy enforcement and pagination.

9

Advanced features

Messaging, stories, ranking and other features only after core foundations work.

This is a dependency-oriented starting order, not a rigid sequence. For instance, media processing can be developed in parallel with some social-graph work once the relevant contracts are stable.

Deliverable: A milestone-based backlog with dependencies, priorities and acceptance criteria.

Exit condition: You know what to implement first, why it comes first and what must be completed before dependent features begin.

Phase 7 — Build one complete vertical slice

This is where coding begins in earnest.

Your first slice should prove that your architecture actually works.

For example, implement publish a post end to end:

```
User action
    ↓
Frontend
    ↓
API authentication and validation
    ↓
Application use case
    ↓
Domain rules and authorization
    ↓
Database transaction
    ↓
Persist post metadata and event
    ↓
Background processing
    ↓
Media ready / publish state updated
    ↓
Frontend reflects the result
```

The actual sequence depends on your upload and publication contract; large media uploads may go directly to object storage using a controlled upload flow.

For this slice, implement:

- Domain logic and authorization.
- Database constraints and migrations.
- API contracts and error handling.
- Transactional behavior and retry safety.
- Background processing where needed.
- Unit and integration tests.
- An end-to-end test.
- Logs, metrics and request correlation.
- A reproducible local development environment.

Then deliberately test failures: invalid input, expired authorization, concurrent publication attempts, worker crashes, duplicate jobs and storage failures.

Deliverable: One fully working, tested and observable feature—not merely a screen and a successful API response.

Exit condition: The feature works across the full stack and behaves predictably under its defined failure scenarios.

Phase 8 — Repeat the vertical-slice workflow

For every subsequent capability, follow the same loop:

1. Review requirements and unknowns
2. Update domain model and contracts
3. Implement domain and application logic
4. Implement persistence and infrastructure
5. Connect API and frontend
6. Test correctness and failure behavior
7. Measure and observe
8. Update documentation and ADRs

Repeat for each capability. Feed discoveries back into the architecture and requirements.

Do not let the implementation drift away from the specification. When a new requirement invalidates an existing decision, update the model and tests deliberately instead of stacking a workaround on top of the old design.

Phase 9 — Implement cross-cutting production concerns

As the core capabilities mature, harden the entire system.

Area	What to implement
Security	Secure sessions, authorization, input validation, secret management, abuse controls
Reliability	Timeouts, bounded retries, idempotency, graceful degradation
Data integrity	Transactions, constraints, concurrency control, migrations
Async processing	Durable job handling, retries, deduplication, dead-letter handling when needed
Observability	Structured logs, metrics, traces, dashboards and alerts
Performance	Query optimization, pagination, caching where justified, load testing
Delivery	CI/CD, environment configuration, deployment checks and rollback
Recovery	Backups, restore drills, retention and disaster recovery
Maintainability	Module boundaries, architecture tests, documentation and dependency management

These concerns should begin in the foundation and golden slice, then mature as the system grows. They are not a final layer of polish.

For example, do not wait until Phase 9 to add authentication or basic validation. Phase 9 means ensuring the overall system’s controls and operational guarantees are comprehensive.

Phase 10 — Validate production behavior

Now verify that the system meets its quality requirements.

Test at multiple levels:

- Unit tests: domain rules and state transitions.
- Integration tests: database, transactions, storage and workers.
- Contract tests: API and event compatibility.
- End-to-end tests: important user journeys.
- Security tests: unauthorized access, abuse and common vulnerabilities.
- Concurrency tests: simultaneous likes, follows, edits and retries.
- Failure tests: unavailable dependencies, timeouts and worker crashes.
- Load tests: realistic traffic, data sizes and concurrent users.
- Migration tests: upgrading representative existing data.
- Recovery tests: restoring data and resuming operations after failure.

Define measurable service-level objectives (SLOs) based on the intended workload.

For example, choose a target for feed latency, API availability and background-job completion, then test under a documented workload. Do not claim that a system is production-grade simply because it passes unit tests.

For a social platform, explicitly validate:

- Private content cannot leak into unauthorized feeds or search results.
- Follow and block operations behave correctly under concurrency.
- Duplicate requests do not create unintended duplicate effects.
- Media access respects authorization and deletion rules.
- Failed background jobs can recover without corrupting state.
- Deployments and schema migrations do not break supported clients.

Deliverable: A quality report showing measured results, known limitations, unresolved risks and release blockers.

Exit condition: Critical behavior is validated against explicit correctness, security, reliability and performance requirements.

Phase 11 — Deploy and operate the system

Treat deployment as part of the product, not an afterthought.

Implement the delivery workflow:

```
Commit
  ↓
CI: lint, tests, security checks, build
  ↓
Versioned artifact
  ↓
Staging deployment
  ↓
Smoke tests and migration validation
  ↓
Production deployment
  ↓
Health checks and SLO monitoring
  ↓
Rollback or mitigation if necessary
```

Prepare:

- Reproducible infrastructure and environment configuration.
- Managed secrets and access controls.
- Database backups and verified restoration.
- Health checks and alerts.
- Deployment and rollback procedures.
- Incident response and operational runbooks.
- A process for safely evolving database schemas and API contracts.

Deploy to a real environment, observe its behavior, and fix the operational issues that testing did not expose.

Deliverable: A deployed, monitored system that another engineer can operate using documented procedures.

Exit condition: You can deploy, diagnose, recover and update the system without depending on undocumented manual steps.

Phase 12 — Final parity review and continuous improvement

Compare your system against the original specification, not against your memory of the reference UI.

For every in-scope requirement, determine whether it is:

- Implemented and verified.
- Implemented but insufficiently tested.
- Partially implemented.
- Not implemented.
- Blocked by an unresolved decision.

Then review your non-functional requirements and known trade-offs.

Ask:

- Are the core workflows correct?
- Are privacy and authorization enforced consistently?
- Are data ownership and consistency boundaries clear?
- Can failures be diagnosed and recovered from?
- Does performance meet the stated targets?
- Can new features be added without widespread changes?
- Are deployment and data migrations repeatable?
- Are limitations documented honestly?

Only declare the project complete relative to its explicit scope and acceptance criteria. You can have a robust, production-oriented implementation without reproducing every feature or the massive operational scale of Instagram.

The exact workflow you should follow throughout the project

The phases above define the lifecycle. This is the operating process you should follow every day.

1. Before every feature

```
Understand the user journey
        ↓
Review evidence and requirements
        ↓
Identify domain rules and invariants
        ↓
Check data model and ownership
        ↓
Define API and failure contracts
        ↓
Confirm architectural fit
        ↓
Write acceptance tests
```

If you discover a missing requirement, resolve it before making arbitrary implementation decisions.

2. During implementation

```
Implement a small, coherent change
        ↓
Run unit tests
        ↓
Run integration tests
        ↓
Review architectural boundaries
        ↓
Test errors and edge cases
        ↓
Run relevant end-to-end tests
        ↓
Update documentation
```

Commit small, reviewable changes. Avoid implementing multiple large, unrelated features simultaneously.

3. At the end of every milestone

```
Validate acceptance criteria
        ↓
Run regression tests
        ↓
Review security and reliability
        ↓
Measure performance where relevant
        ↓
Review technical debt and unknowns
        ↓
Update architecture documentation
        ↓
Approve the next milestone
```

If the architecture needs to change, record why and update the relevant contracts and tests.

Your project repository

Maintain two separate repositories or clearly separated areas: one for research and one for your implementation.

Reference research

Your evidence and reconstructed specification

```
reference-research/
├── 01-product-inventory/
├── 02-user-journeys/
├── 03-requirements/
├── 04-domain-model/
├── 05-state-machines/
├── 06-api-observations/
├── 07-failure-behavior/
├── 08-architecture-hypotheses/
├── 09-open-questions/
└── 10-acceptance-criteria/
```

Your implementation

The independently designed and tested system

```
social-platform/
├── apps/
│   ├── web/
│   └── api/
├── modules/
│   ├── identity/
│   ├── profiles/
│   ├── social-graph/
│   ├── posts/
│   ├── feed/
│   └── notifications/
├── workers/
├── migrations/
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── contract/
│   ├── e2e/
│   ├── load/
│   └── failure/
├── infrastructure/
├── deploy/
└── docs/
    ├── architecture/
    ├── decisions/
    ├── security/
    └── operations/
```

These are starting structures, not mandatory folder layouts. Adapt them to your language, framework and deployment model.

The most important rules to remember

1. Never equate screens with features or features with systems. A screen is one interface to a collection of domain rules, data and workflows.

2. Never start a major feature without understanding its requirements. Unresolved uncertainty is fine when documented; hidden assumptions are dangerous.

3. Design boundaries before implementation, then validate them through real code. Neither endless upfront design nor uncontrolled coding works well.

4. Prefer a complete vertical slice over many incomplete features. Prove the architecture early.

5. Make correctness the foundation. Authorization, transactions, invariants and data integrity cannot be replaced with a better UI.

6. Design for failure explicitly. Retries, timeouts, concurrency, duplicate requests and partial failures are normal production concerns.

7. Do not add infrastructure without a reason. Every database, queue, cache and service introduces operational cost.

8. Keep observed facts separate from architectural hypotheses. You are reconstructing a system from evidence, not magically recovering its source code.

9. Treat testing, observability, security and operations as part of development. They are not optional final steps.

10. Optimize for verified quality, not feature count. A smaller system that is correct, secure, maintainable and recoverable is a much stronger foundation.

What you should do first, starting today

Don’t begin by implementing Instagram’s UI. Complete this first set of tasks in order.

Starting checklist

0 of 9

1. Create the research repository

Set up folders for observations, workflows, requirements and unknowns.

2. Write the project charter

Define the target product, initial scope, constraints and non-goals.

3. Map the product capabilities

Inventory accounts, profiles, posts, follows, feeds, likes, comments and notifications.

4. Document five critical journeys

Start with signup/login, publish a post, follow a user, like/comment and view the feed.

5. Investigate edge cases

Study permissions, retries, privacy, failures, concurrent actions and empty states.

6. Write the initial requirements

Turn the observations into testable expected behavior.

7. Create the first domain model

Define entities, relationships, invariants and state transitions.

8. Design the initial architecture

Draw the components, ownership boundaries, data flows and failure paths.

9. Define the first vertical slice

Choose one workflow, its contracts, tests and measurable acceptance criteria.

Copy checklist

Once these tasks are complete, begin implementing the first vertical slice. Do not wait until every future requirement or architecture decision is known; design enough to make the first slice sound, then learn and refine through implementation.

Your complete development loop is:

Study → Specify → Model → Architect → Contract → Plan → Implement → Test → Harden → Deploy → Operate → Validate → Iterate.

That is the repeatable process you should apply to every production system you reconstruct, whether it is a social platform, e-commerce application, project-management tool or another complex product.